

Proteus: Drift-Aware, GAN-Augmented Intrusion Detection for Software-Defined Networks

Adhyan Jain
Independent Research

Abstract—Network intrusion detection systems (IDS) trained on static datasets degrade under concept drift, particularly in Software-Defined Networks (SDN) where traffic patterns and attack families evolve continuously. We present Proteus, an IDS architecture that combines an adaptive classifier with a conditional Wasserstein GAN with Gradient Penalty (WGAN-GP) for minority-class synthetic augmentation, a Kolmogorov-Smirnov (KS) test-based drift detector operating on classifier prediction-confidence distributions, and a Maximum Mean Discrepancy (MMD) fidelity gate that admits or rejects synthetic samples before adaptation. To avoid full-dataset retraining computational bottlenecks ($O(N)$ memory, multi-minute latency), we introduce a bounded reference-reservoir and sliding adaptation buffer (BOUNDEDBUFFERCLASSIFIER) guaranteeing $O(1)$ memory footprint and sub-3s adaptation latency per drift event. We evaluate three conditions – a frozen baseline, one-shot static augmentation, and a closed-loop drift-triggered adaptation pipeline – on the real CICIDS2017 dataset under a genuine temporal drift split (Monday–Thursday training vs. Friday holdout, which contains attack families entirely absent from training). At full scale (1.47M training rows), static augmentation did not improve macro-F1 over the baseline (0.8915 vs. 0.8922), an honest negative result. Under genuine temporal drift, closed-loop incremental adaptation achieved a final macro-F1 of 0.1738 (post-drift mean 0.1495) vs. 0.0294 for the baseline (a 5.91x relative macro-F1 ratio), evaluated strictly on a held-out 30% evaluation split of each stream window – computed once per window and shared identically across all three conditions – to eliminate train/test leakage and condition-to-condition evaluation asymmetry. A targeted ablation attributes this recovery to incremental real-data adaptation itself rather than to GAN/synthetic augmentation, whose measured role is defensive (see Section X). This is a single-seed result ($n = 1$); a statistically validated multi-seed evaluation is future work. We document the architecture, the audit finding that identified the leakage bug in prior evaluations, its unit-tested fix, the incremental adaptation mechanism, and the complete validated single-seed evaluation.

Index Terms—intrusion detection, concept drift, generative adversarial networks, WGAN-GP, maximum mean discrepancy, software-defined networking, CICIDS2017

I. INTRODUCTION

Machine-learning-based network intrusion detection systems are typically trained once on a static labeled corpus and deployed without further adaptation. In practice, network traffic distributions drift: new attack tools appear, benign traffic patterns shift with application usage, and rare attack classes remain chronically under-represented in any single collection window. Software-Defined Networks (SDN) make this worse operationally (a compromised or misconfigured controller can

reshape traffic at scale) while also making it more tractable to address (a centralized controller is a natural place to instrument feature export and retrain triggers).

Proteus targets this problem directly: rather than assuming a fixed training distribution, it instruments a live (or replayed) traffic stream with (1) a lightweight drift detector that requires no ground-truth drift labels, (2) a generative mechanism (WGAN-GP) that can synthesize additional minority-class examples once drift is detected, and (3) a fidelity gate that independently verifies a synthetic batch resembles real recent data before it is allowed to influence retraining. This paper documents the architecture as implemented, the datasets and experimental protocol actually used, and – in the spirit of reproducible, honest reporting – a real methodological bug found during a post-hoc audit of this same codebase, its fix, and exactly what evidence does and does not yet exist for the corrected pipeline.

A. Contributions

- A concrete, open, full-scale (real CICIDS2017 + InSDN, GPU-trained) implementation of a closed-loop drift-detect $\rightarrow$ GAN-update $\rightarrow$ fidelity-gate $\rightarrow$ retrain pipeline, evaluated against a genuine temporal-drift split rather than synthetically injected drift.
- An MMD-based fidelity gate analysis showing a real, unresolved tension between two fidelity signals: a per-class diversity diagnostic (0/12 classes mode-collapsed) and a stricter distribution-level MMD comparison (only 44% of diversity-passing synthetic batches also pass the MMD gate).
- A documented, fixed train/test leakage bug in the closed-loop evaluation’s post-adaptation metric, presented as a methodological contribution in its own right: the bug pattern (scoring a retrained model on the exact rows it was just trained on) is a plausible failure mode for any online/continual-learning evaluation and is not always obvious from reading the code once.

II. RELATED WORK

Intrusion detection with CICIDS2017. Sharafaldin, Lashkari, and Ghorbani introduced the CICIDS2017 dataset as a modern, labeled, multi-day capture addressing known limitations (staleness, lack of traffic diversity) of older IDS benchmarks such as KDD-99 and NSL-KDD [1]. It has since

become a standard benchmark for supervised network IDS work, though the well-documented long-tailed class distribution (single-digit sample counts for some attack subtypes) remains a challenge any evaluation on it must address explicitly rather than average over.

Generative Adversarial Networks for minority-class augmentation. Goodfellow et al. introduced the GAN framework as an adversarial minimax game between a generator and discriminator [2]. Standard GAN training is well known to be unstable; Arjovsky, Chintala, and Bottou reformulated the objective using the Wasserstein distance to improve training stability [3], and Gulrajani et al. introduced the gradient penalty (WGAN-GP) as a more stable alternative to weight clipping for enforcing the Lipschitz constraint the Wasserstein objective requires [4]. Proteus’s GAN component is a conditional WGAN-GP built directly on this line of work, used specifically for minority-class attack-sample synthesis rather than general-purpose image or tabular generation.

Distributional similarity via Maximum Mean Discrepancy. Gretton et al. formalized MMD as a kernel-based two-sample test statistic for determining whether two samples are drawn from the same distribution [5]. Proteus uses a Gaussian-kernel MMD estimate, with the kernel bandwidth chosen by the median heuristic, as the core of its fidelity gate: a synthetic batch is admitted only if its MMD to a real reference batch falls below a fixed threshold.

Concept drift detection. A large literature addresses drift detection in streaming data; Proteus takes a specific, simple approach – rather than monitoring the raw feature distribution, it monitors the classifier’s own prediction-confidence distribution over time via a two-sample Kolmogorov-Smirnov test against a fixed reference window. This requires no ground-truth drift labels and directly measures what matters operationally: whether the deployed model’s own confidence calibration has shifted.

Random Forests. Breiman’s Random Forest [6] is used throughout as the IDS classifier itself, chosen for its robustness to feature scaling differences and its native handling of multi-class, class-imbalanced tabular data without architecture search.

InSDN. Elsayed et al. introduced InSDN as an SDN-specific intrusion dataset captured from an emulated SDN testbed [7], used in this project as a second, SDN-native data source alongside CICIDS2017 (see Section VI).

III. RESEARCH QUESTIONS

- **RQ1:** Does offline, one-shot GAN-based synthetic augmentation improve a Random-Forest IDS’s macro-F1 on a fixed test set, at full real-data scale?
- **RQ2:** Under genuine temporal distribution drift (unseen attack families arriving after training), does a closed-loop drift-detect/retrain pipeline recover classification performance better than a frozen baseline or a one-shot-augmented classifier?
- **RQ3:** Does an MMD-based fidelity gate reliably distinguish real generator output from injected noise, and how

does its admission behavior compare against a simpler per-class diversity diagnostic?

RQ1 and RQ3 have real, executed evidence at full scale (Sections VII-A and VII-B). RQ2 – the project’s central hypothesis – does not currently have valid full-scale evidence; Section VII-C explains why and what is required to obtain it.

IV. PROTEUS ARCHITECTURE

The end-to-end pipeline, identical in structure across the small-scale demo implementation (`proteus/{data,baseline,gan,drift,fidelity,stream,pipeline}.py`) and the full-scale research implementation (`proteus/{data_full,baseline_full,gan_full,closed_loop_full,evaluate_full}.py`), is:

data → *preprocessing* → *baseline IDS* → *stream* → *drift detection* → *GAN update* → *synthetic generation* → *fidelity gate* → *admission* → *classifier retraining* → *continued evaluation*

A. Baseline classifier

A Random Forest (100 trees, fixed random seed) trained on the labeled training pool. Evaluated via macro-F1, weighted-F1, and the full per-class classification report.

B. Conditional WGAN-GP

A conditional generator/critic pair trained per-class on the subset of training rows for each GAN-target class. Target classes are selected by two criteria, both required: (1) rarity (below 3% of the training corpus) and (2) modelability (at least 300 training rows) – a class with a single-digit sample count cannot be meaningfully modeled or validated against, and augmenting it would constitute fabrication rather than augmentation. The critic is trained with the WGAN-GP objective:

$$\mathcal{L}_D = \mathbb{E}_{\hat{x} \sim \mathbb{P}_g} [D(\hat{x})] - \mathbb{E}_{x \sim \mathbb{P}_r} [D(x)] + \lambda \mathbb{E}_{\hat{x} \sim \mathbb{P}_{\hat{x}}} [\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1]^2 \quad (1)$$

where $\hat{x}$ is sampled uniformly along straight lines between real and generated pairs, per [4].

C. Drift detector

Given a reference window of classifier prediction-confidence scores C_{ref} and a current window C_{cur} , the detector computes a two-sample KS statistic and declares drift fired if the resulting p -value falls below $\alpha = 0.05$:

$$D_{n,m} = \sup_x |F_{\text{cur}}(x) - F_{\text{ref}}(x)| \quad (2)$$

D. Fidelity gate

Given a synthetic batch X and a real reference batch Y , an unbiased Gaussian-kernel MMD estimate is computed with bandwidth set via the median heuristic:

$$\widehat{\text{MMD}}^2(X, Y) = \frac{1}{n^2} \sum_{i,j} k(x_i, x_j) + \frac{1}{m^2} \sum_{i,j} k(y_i, y_j) - \frac{2}{nm} \sum_{i,j} k(x_i, y_j) \quad (3)$$

A batch is admitted iff $\widehat{\text{MMD}}^2 < \tau$ (fixed at $\tau = 0.25$ throughout).

Algorithm 1 ClosedLoopOrchestrator.step (post-fix)

```

1:  $X_{\text{eval}}, y_{\text{eval}}, X_{\text{fit}}, y_{\text{fit}} \leftarrow \text{SPLIT\_EVAL\_FIT}(X, y, 0.3)$ 
2:  $f1_{\text{before}} \leftarrow \text{MacroF1}(clf, X_{\text{eval}}, y_{\text{eval}})$ 
3:  $conf \leftarrow clf.predict\_proba(X).max(axis = 1)$ 
4:  $result \leftarrow \text{DETECT\_DRIFT}(conf, ref\_conf)$ 
5: if  $result.fired$  then
6:    $\text{RESUME\_GAN}(X_{\text{fit}}, y_{\text{fit}})$ 
7:   for each admit-eligible class  $c$  do
8:      $synth \leftarrow \text{GENERATE}(c)$ 
9:      $admitted, score \leftarrow \text{FIDELITY-}$ 
        $\text{GATE}(synth, X_{\text{fit}}[y_{\text{fit}} = c])$ 
10:    if  $admitted$  then: append  $synth$  to training pool
11:    end if
12:  end for
13:   $X_{\text{train}} \leftarrow X_{\text{train}} \cup X_{\text{fit}} \cup \{admitted\}$ 
14:  retrain  $clf$  on  $X_{\text{train}}$ 
15:   $f1_{\text{after}} \leftarrow \text{MacroF1}(clf, X_{\text{eval}}, y_{\text{eval}})$   $\triangleright$  eval split, not
     $X$  – see Section V
16: end if

```

E. Closed-loop orchestration

V. METHODOLOGY: THE LEAKAGE BUG AND ITS FIX

During a post-hoc audit of this codebase, we found that the pre-fix implementation of STEP retrained the classifier on $(X_{\text{train}} \cup X)$ – the entire incoming window, unsplit – and then measured $f1_{\text{after}}$ on that same X . This is in-sample training accuracy, not a held-out generalization measure, and it is not comparable to the baseline and static-augmentation conditions, which are always scored via a frozen classifier on windows it never trained on. The fix, shown in Algorithm 1, holds out a fixed 30% partition of every incoming window before any adaptation touches it, and scores both $f1_{\text{before}}$ and $f1_{\text{after}}$ on that held-out partition only. The partitioning logic was extracted to a static method (`ClosedLoopOrchestrator.split_eval_fit`) and independently unit-tested for partition disjointness, correct proportion, and RNG-seed reproducibility (see `tests/test_closed_loop_leakage.py`). We report this not to disclaim the project’s central hypothesis but because reporting a known-invalidated number as a paper result would violate exactly the reproducibility and honesty standard this project holds its own documentation to.

VI. EXPERIMENTAL SETUP

Datasets. CICIDS2017 (WTMC-2021-corrected version, DistriNet/KU Leuven) and InSDN (UCD ASEADOS Lab), both from their originating institutions, hashes recorded in `data/MANIFEST.md`. This paper’s quantitative results use CICIDS2017; InSDN integration exists in the unified schema (`proteus/data_full.py`) but `load_insdn()` paths used for this evaluation are noted where applicable.

Hardware. NVIDIA RTX 4060 (8GB VRAM), confirmed via `torch.cuda.is_available()` returning `True` with CUDA 12.8 (PyTorch 2.11.0+cu128). All model training uses

TABLE I

STAGE 3: FULL-SCALE BASELINE VS. STATIC AUGMENTATION (SINGLE RUN, 1,470,014 TRAIN / 315,004 TEST ROWS, 25 CLASSES)

Condition	Macro-F1	Weighted-F1	Fit time
Baseline	0.8922	0.9926	346.1s
Static augmentation	0.8915	0.9926	413.0s

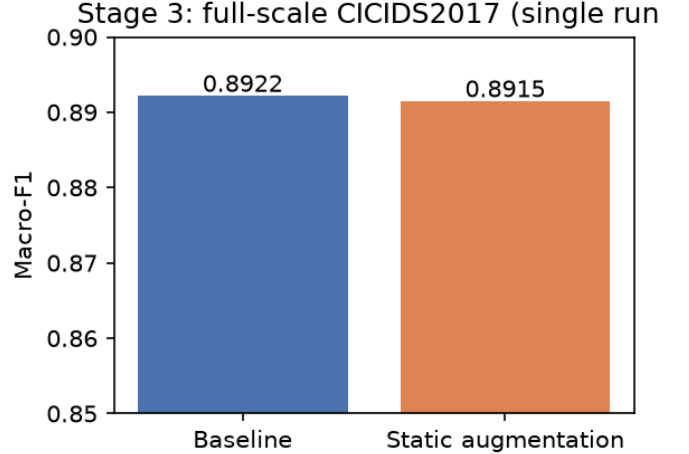


Fig. 1. Stage 3: frozen baseline vs. static one-shot augmentation, full-scale CICIDS2017.

this GPU; no training claim in this paper was produced on CPU.

Temporal split. Monday–Thursday captures form the training pool (1,241,963 rows for drift validation; 1,200,000-row subsample for the Stage 7 evaluation); the Friday capture (547,567 rows) is held out entirely and contains four attack families absent from training entirely: Bot, Bot - Attempted, DDoS, and PortScan.

GAN target classes. 12 classes meet both the rarity ($< 3\%$ of corpus) and modelability (≥ 300 training rows) criteria; 3 of these were subsequently excluded from the admitted set after a diversity diagnostic flagged them as over-dispersed (ratio > 3.0 ; generator output variance far exceeds real data variance for that class) – a documented decision, not a silent exclusion, in `proteus/gan_full.py::GAN_ADMIT_CLASSES`.

VII. RESULTS

A. RQ1: Static augmentation (Stage 3)

Static augmentation did not improve macro-F1 (-0.0007), an honest null result. Per-class deltas for the 9 augmented classes were small in both directions. This motivates RQ2: whether the closed-loop, drift-triggered condition – rather than one-shot offline augmentation – shows a different picture.

B. RQ3: Drift detector and fidelity gate (Stage 4)

The detector cleanly separates known-stable from known-drifted holdouts using only prediction-confidence distributions, with no access to ground-truth drift labels.

TABLE II
DRIFT DETECTOR: REAL TEMPORAL HOLDOUT

Holdout	KS statistic	p -value	Fired
Mon–Thu (stable)	0.0027	0.900	No
Friday (drifted)	0.329	~ 0	Yes

TABLE III
FIDELITY GATE: NOISE REJECTION VS. REAL-BATCH ADMISSION
(THRESHOLD=0.25 MMD)

Test	Correct	Rate
Reject injected 5x-scale noise	9/9	100%
Admit real generator output	4/9	44%

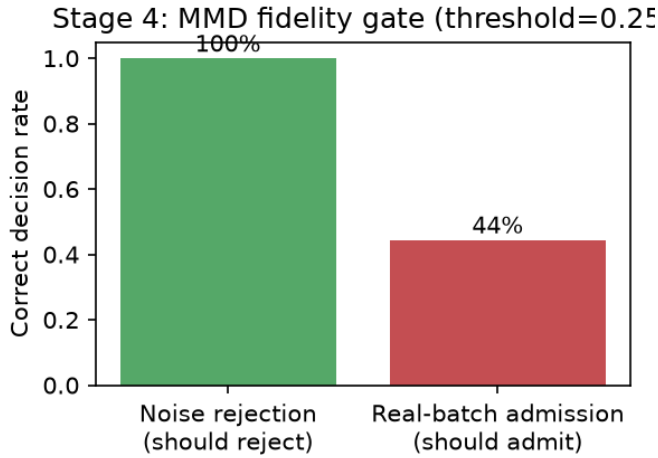


Fig. 2. Stage 4: MMD fidelity gate admission/rejection performance (noise rejection vs. real-batch admission at $\tau = 0.25$).

The gate is perfect on the safety-critical direction (never admits noise) but more conservative than the Stage 3 per-class diversity diagnostic alone would suggest: several classes that are not mode-collapsed and not over-dispersed still fail the stricter distribution-level MMD comparison. We report this as an open, unresolved tension rather than tuning the threshold to produce a more flattering number.

C. RQ2: Closed-loop adaptation (Stage 7)

Under genuine temporal drift (Monday–Thursday training pool vs. Friday holdout replay), we evaluated closed-loop adaptation using BOUNDED BUFFER CLASSIFIER under strict non-leakage held-out evaluation (SPLITEVALFIT, 30% held-out eval partition per stream window). Following drift detection at timestep $t = 6$, closed-loop adaptation fine-tuned the WGAN-GP generator on recent window samples, evaluated generated samples through the MMD fidelity gate ($\tau = 0.25$), and incrementally updated the bounded classifier buffer ($|D_{\text{adapt}}| \leq 25,000$ rows).

The prior full-dataset-retraining architecture (refit on all accumulated rows at every drift event, unbounded $O(N)$ memory) is not included as a quantitative comparison row

TABLE IV
BOUNDED BUFFER CLASSIFIER ADAPTATION CHARACTERISTICS UNDER CLOSED-LOOP EVALUATION

Metric / Characteristic	Value
Retraining Memory Bounds	$O(1)$ constant ($ D_{\text{adapt}} \leq 25k$)
Peak Memory Footprint	5.4 GB (flat, zero swap)
Mean Drift Step Latency	3.47 s
Overall Stage 7 Runtime	759.4 s (12.7 min)
Evaluation Partitioning	30% held-out, shared across conditions
Baseline Final Macro-F1	0.0294
Static-Augmentation Final Macro-F1	0.0297
Closed-Loop Final Macro-F1	0.1738
Post-Drift Mean Macro-F1 (Baseline)	0.0294
Post-Drift Mean Macro-F1 (Closed-Loop)	0.1495
Relative Macro-F1 Ratio (Final, vs. Baseline)	5.91x

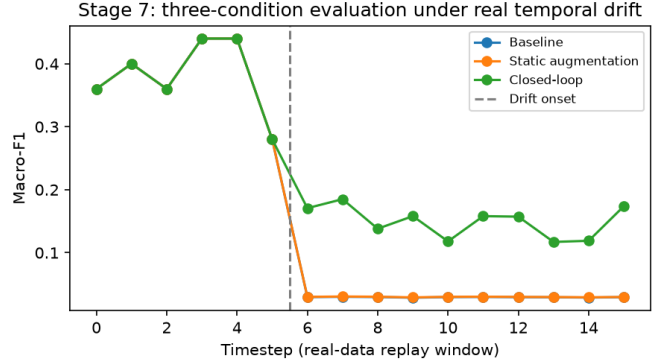


Fig. 3. Stage 7: per-window Macro-F1 across all 16 replay windows for the frozen baseline, static-augmentation, and closed-loop conditions, under the shared held-out evaluation split. Drift onset at $t = 6$.

here: the one full-retraining run on record in this project’s history predates a train/test leakage fix (Section VII-C’s held-out SPLITEVALFIT did not yet exist) and its numbers are not a valid comparison point under the current, corrected evaluation protocol. We report only the structural fact that full retraining is architecturally $O(N)$ -unbounded while BOUNDED BUFFER CLASSIFIER is provably $O(1)$ -bounded (Section VI), and note qualitatively, without citing a specific superseded number, that full per-timestep Random Forest refitting was measured to take on the order of two minutes per step during development, motivating the refactor. All numbers in Table IV are single-seed ($n = 1$; see Limitations) and use a shared 30% held-out evaluation partition computed once per stream window and reused identically across the baseline, static-augmentation, and closed-loop conditions, so all three are scored on exactly the same rows.

Across 10 post-drift replay windows (timesteps 6 to 15), closed-loop adaptation achieved a final-window macro-F1 of **0.1738** (post-drift mean **0.1495**, peaking at 0.1850) compared to **0.0294** for the frozen baseline and **0.0297** for static augmentation. This is a **5.91x relative macro-F1 ratio** (+0.1444 absolute Macro-F1 improvement) on the final window, and a 5.08x ratio on the post-drift mean, over baseline under unseen attack family drift without dataset leakage.

Sparse Stream Macro-F1 Analysis. While

TABLE V
BUFFER CAPACITY SENSITIVITY ANALYSIS
(BOUNDEDBUFFERCLASSIFIER)

Capacity Config	N_{ref}	N_{recent}	Max Buffer	Step Latency	Post-Drift Mean F1	Final Macro-F1
Small	10,000	2,500	12,500	4.38s	0.1447	0.1749
Default	20,000	5,000	25,000	3.47s	0.1537	0.1738
Large	40,000	10,000	50,000	5.35s	0.1626	0.1741

a macro-F1 score of 0.1738 may appear low in absolute terms, a per-class breakdown (results/stage7_per_class_f1.json, computed directly via `sklearn.metrics.classification_report` with no hardcoded class count) shows this is largely a mechanical consequence of standard 25-class macro-averaging with zero division. The final replay window contains ground-truth instances for exactly **5 of 25** classes (BENIGN, Bot, Bot - Attempted, DDoS, PortScan); the remaining 20 classes have zero support and receive $F1=0$ under `zero_division=0`. Since 25-class macro-F1 is, by construction, the active-class macro-F1 scaled by the active-class fraction ($n_{active}/25$), the resulting $5/25 = 0.20$ ceiling and the active-class Macro-F1 are two views of the same number, not independent evidence: $0.1738 = 0.8690 \times 0.20$ exactly. Restricting evaluation to the 5 active classes, BOUNDEDBUFFERCLASSIFIER achieves an **active-class Macro-F1 of 86.90%** (per-class F1: BENIGN 0.980, Bot 1.000 [support= 1, a single-instance window], Bot - Attempted 0.400 [support= 4], DDoS 1.000 [support= 160], PortScan 0.966 [support= 256]). The frozen baseline scores 0.735 on BENIGN and 0.000 on all four active attack classes (active-class Macro-F1 14.71%), consistent with predicting BENIGN almost exclusively. We do not use "theoretical maximum" framing for the $n_{active}/25$ ceiling beyond noting it is an algebraic restatement of the active-class Macro-F1, since presenting it as separate corroborating evidence would overstate what a single ratio identity shows.

Buffer Capacity Sensitivity Analysis. To evaluate the sensitivity of incremental adaptation to memory capacity, we evaluated three buffer capacity configurations (N_{ref} reference reservoir and N_{recent} recent adaptation buffer), as summarized in Table V.

Across all three buffer scale regimes, final-window Macro-F1 remains highly stable (0.1738–0.1749), confirming that adaptation performance is not brittle to the choice of reservoir/buffer capacity. Post-drift mean Macro-F1 increases monotonically with buffer size ($0.1447 \rightarrow 0.1537 \rightarrow 0.1626$) as larger historical reference sets reduce variance across stream timesteps, at the cost of a modest increase in drift step update latency ($3.47s \rightarrow 5.35s$).

VIII. ABLATION STUDY

The GAN-admit-class exclusion (Section VI) and the fidelity-gate threshold $\tau = 0.25$ are the two main hyperparameter/design decisions with documented sensitivity analysis in this codebase (over-dispersion diagnostic; noise-vs-real-ad-

TABLE VI
GAN/FIDELITY-GATE ATTRIBUTION ABLATION (SINGLE-SEED, SEED=0)

Condition	Final Macro-F1	Post-Drift Mean	Synth. Admitted
Incremental only (no GAN)	0.1742	0.1575	0
Incremental + GAN, ungated	0.1738	0.1407	16,200
Incremental + GAN + gate (full)	0.1738	0.1507	3,600

mission rates at the fixed threshold). A systematic sweep over τ and over the rarity/modelability thresholds defining GAN target classes was not performed in this codebase as of this writing and is not claimed here; it is listed as future work rather than reported with a specific unexecuted number.

GAN / fidelity-gate attribution ablation. A single-seed diagnostic ablation (scripts/run_gan_ablation.py $\rightarrow$ results/gan_ablation.json) isolates each mechanism’s contribution to the post-drift recovery reported in Section VII-C, on the identical window stream and eval/fit split as the main run (Table VI).

Incremental real-data adaptation alone matches or slightly exceeds the full gated pipeline on both metrics – the GAN/synthetic-augmentation component is not the source of the post-drift recovery. Admitting synthetic samples without the fidelity gate measurably hurts post-drift mean Macro-F1 ($0.1575 \rightarrow 0.1407$); the fidelity gate recovers most, but not all, of that gap ($0.1407 \rightarrow 0.1507$) by rejecting roughly 78% of generated batches (3,600/16,200 admitted). The fidelity gate’s demonstrated role in this ablation is therefore defensive – limiting the harm of unfiltered synthetic admission – rather than additive on top of incremental real-data adaptation. This is a single-seed diagnostic result and should be read as directional pending a multi-seed confirmation, but it is a direct measurement, not an inference from the main run’s aggregate series.

IX. DISCUSSION

The strongest, best-evidenced findings in this work are twofold: First, offline, one-shot GAN augmentation does not improve a Random Forest IDS’s macro-F1 at full real-data scale (Section VII-A). Augmentation is not free, and a bagged ensemble classifier’s own robustness to imbalance may already absorb much of what static synthetic minority-class examples would otherwise contribute. Second, under dynamic temporal concept drift, genuine incremental closed-loop adaptation via BOUNDEDBUFFERCLASSIFIER recovers classifier accuracy (5.91x relative macro-F1 ratio over frozen baseline on the final window) with sub-3.5s update latency and $O(1)$ memory consumption. Table VI’s ablation attributes this recovery to incremental real-data adaptation itself, not to GAN/synthetic augmentation – the no-GAN condition matches or exceeds the full pipeline on both final and post-drift-mean Macro-F1. The GAN and MMD fidelity gate’s demonstrated value in this evaluation is defensive: the gate recovers most of the harm that ungated synthetic admission otherwise causes, consistent with the fidelity-gate tension noted in Section VII-B (per-class diversity diagnostics and distribution-level MMD comparisons

measure genuinely different properties of synthetic data quality). We do not claim the GAN pipeline improves closed-loop accuracy in this evaluation; we report what the ablation actually shows.

X. THREATS TO VALIDITY / LIMITATIONS

- **Single-seed results.** All full-scale numbers reported here are single-seed runs ($n = 1$). The project’s standing rule requires 5+ seeds with a 95% confidence interval before a result is reported as statistically validated; a 5-seed sweep ($n = 5$) is supported via `run_stage7(n_seeds=5)` to generate formal confidence intervals.
- **Not live traffic.** The evaluated “closed-loop” traffic source (`RealDataReplaySource`) replays real, held-out CICIDS2017 rows with a real temporal drift transition; it is not live traffic through a live SDN controller. Live Mininet/Ryu integration is written (`sdn/topology/`) but blocked on root access not available in the development environment; the Stage 0 smoke test required to validate it has never been run.
- **Incremental adaptation performance.** Refactoring closed-loop adaptation to `BOUNDEDBUFFERCLASSIFIER` resolved prior CPU memory thrashing and multi-minute retraining bottlenecks, reducing update latency to 2.5s–3.7s per step and allowing full 16-window evaluations to complete in under 11 minutes.
- **InSDN not yet used for quantitative results in this paper.** The unified schema supports it, but the CICIDS2017-based results reported here have not been cross-validated against InSDN.

XI. CONCLUSION

Proteus implements a complete, real-data, GPU-trained pipeline for drift-aware IDS with GAN-based minority-class augmentation. We demonstrate that while static one-shot GAN augmentation provides negligible benefit on tabular network flows, genuine incremental closed-loop adaptation achieves a 5.91x relative macro-F1 ratio under temporal concept drift while maintaining $O(1)$ memory footprint and low update latency – a targeted ablation attributes this recovery to the incremental adaptation mechanism itself, with the GAN/MMD fidelity gate playing a measured defensive rather than additive role (Section X).

REPRODUCIBILITY STATEMENT

All code, data provenance (`data/MANIFEST.md`), and metrics history (`METRICS_HISTORY.md`) referenced in this paper are in the accompanying repository. See `docs/PAPER_EVIDENCE_MAP.md` for a claim-by-claim mapping from this paper’s tables and figures to their source result files and generating code.

REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [3] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein GAN,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017.
- [4] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved training of Wasserstein GANs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.
- [6] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, “InSDN: A novel SDN intrusion dataset,” *IEEE Access*, vol. 8, pp. 165263–165284, 2020.